INTERVIEW PREPARATION

Cisco IQ — SRE (CX Engineering)

83 tricky real-world scenario questions across 14 topic areas

---

Role: Site Reliability Engineer, Cisco IQ Team, CX Engineering

Level: 8–14+ years · architectural-ownership scope

Focus: AWS, Kubernetes/EKS, Terraform, Docker, CI/CD, observability (Prometheus/Thanos/Splunk/CloudWatch/OpenTelemetry/ELK), Linux/networking/security, mentoring & customer-facing SRE

Prepared: September 4, 2026

Contents

1. Role & Positioning — 5 questions
2. Architectural Vision & Roadmap — 6 questions
3. SRE Principles: SLI/SLO/Error Budgets — 7 questions
4. Incident Management & Postmortems — 6 questions
5. AWS & Multi-Region Reliability — 7 questions
6. Kubernetes / EKS — Tricky Scenarios — 7 questions
7. Terraform / IaC — Tricky Scenarios — 5 questions
8. Docker & CI/CD Gotchas — 5 questions
9. Observability: Prometheus/Thanos/Splunk/CloudWatch/OTel/ELK — 9 questions
10. Linux, Networking & Security — 6 questions
11. Automation & Programming (Python/Go) — 5 questions
12. Customer & Cross-Functional Engagement — 5 questions
13. Mentoring & Technical Leadership — 4 questions
14. Behavioral — Tricky Situational — 6 questions

Cisco IQ sits in CX Engineering — customer-facing SaaS reliability at multi-region scale. Positioning: lead with AWS hands-on time, frame Azure certs (AZ-104/SC-300/SC-100) as cross-cloud breadth, and use the ownership-depth reframe — "the platform team owned the design, I owned operating/evolving/troubleshooting it" — anywhere you didn't architect something from scratch. Items marked **Bring your own** are frameworks, not fabricated stories — slot in your real Barclays / Russell Investments / Terawe / Unique Vacations / RSAB examples.

1 Role & Positioning

☐ 1. Cisco IQ sits inside CX Engineering — what does that combination signal about the role, and how would you position your background against it?

> CX (Customer Experience) Engineering means this SRE role is embedded in a product/services org directly accountable to paying customers, not a pure internal-platform team — reliability work here gets measured against customer-visible SLAs, support escalations, and renewal risk, not just internal dashboards. Position your background around wherever you've been closest to that same accountability line — regulated, customer/business-facing environments like Barclays and Russell Investments — and lead with an incident where you translated a customer-impacting problem into a technical fix, not just infra you touched. **Bring your own:** pick one incident where the trigger was customer-reported rather than internally-detected, and walk the RCA end-to-end.

☐ 2. The JD lists AWS as 'preferred' but your certifications are Azure (AZ-104/SC-300/SC-100). How do you address that gap directly if asked?

> Reuse the established reframe rather than dodge it: lead with your AWS hands-on years as the primary claim, then position the Azure certifications as evidence of cross-cloud fluency — IAM/identity and security-posture concepts transfer directly between clouds, and dual-cloud experience is increasingly what enterprise SaaS platforms want given multi-cloud commitments and M&A. Don't apologize for the cert mix; frame it as breadth, not a gap.

☐ 3. This is an 8–14+ year band with 'lead the architectural vision' language — how do you show you're ready for that scope, not just execution?

> The ownership-depth reframe applies directly: if you didn't design a given platform from scratch, say who did and what your ownership scope was (operate / evolve / troubleshoot), then pivot to a concrete example where you drove a technical decision end to end — a migration, a tool choice, a runbook standard. Avoid "I built X" claims for infra you didn't architect; "I owned the reliability and evolution of X, including redesigning Y" is both honest and reads as senior.

☐ 4. Why CX Engineering / Cisco IQ specifically, versus a generic platform SRE role?

> **Bring your own** — but the honest answer usually centers on wanting SRE work with direct line-of-sight to customer outcomes rather than purely internal infra, plus interest in the scale (multi-region SaaS) and breadth (architecture + hands-on + mentoring) the JD describes. Tie it to something concrete you're genuinely looking for next, not a generic "I like challenges" line.

☐ 5. The JD asks for CKA/CKAD/AWS DevOps Engineer 'or equivalent' certifications — you don't hold any of the three. How do you handle that if asked directly?

> Don't overclaim. State plainly you don't hold CKA/CKAD, but describe your hands-on Kubernetes/EKS depth as the substitute evidence, and if you're genuinely planning to sit for one, say so as a concrete next step rather than a vague intention. "Or equivalent" language in postings usually means hands-on depth carries real weight against the credential — lean into demonstrated depth in the technical rounds rather than relitigating the cert gap.

2 Architectural Vision & Roadmap

☐ 1. Tell me about a time you had to balance immediate operational needs against a longer-term architectural goal. How did you decide?

> **Bring your own** — framework to hang it on: articulate the trade-off explicitly (toil now vs. debt later), quantify the cost of deferring (on-call load, incident frequency), and describe how you sequenced it — e.g. shipped a stopgap while carving the underlying fix into a tracked initiative rather than letting it disappear.

☐ 2. How do you build a technical roadmap for a platform when priorities are coming from multiple directions — customers, product, security, engineering?

> Start from a shared source of truth for pain (incident data, error-budget burn, support-ticket themes), translate each stakeholder's ask into cost/impact terms so they're actually comparable, then sequence by risk-reduction-per-unit-effort. Treat the roadmap as a living document revisited on a cadence — quarterly, not fixed once at the start of the year.

☐ 3. Give an example — or a framework — for how you'd influence a decision when you don't have direct authority over the team that needs to change.

> Lead with data/incident evidence rather than opinion; find the other team's own incentive (their on-call pain, their SLA risk) and frame the ask in those terms; propose a low-cost pilot rather than a big-bang change; and escalate through architecture review or your manager only after a genuine direct alignment attempt, not as a first move.

☐ 4. How do you decide when a platform problem needs an architectural rewrite versus incremental hardening?

> Incremental hardening when the current design's failure modes are well understood and the fix cost is low relative to risk. Rewrite when the same failure pattern recurs despite hardening — a sign the abstraction itself is wrong — or when scale requirements (multi-region, order-of-magnitude growth) exceed what the current design can be patched to support.

☐ 5. Cisco IQ operates at multi-region scale — walk through your framework for architecting a service for multi-region resilience.

> Cover: active-active vs. active-passive trade-offs; the data replication/consistency model choice and its latency cost; region-aware routing and failover (DNS health-check failover vs. an anycast/accelerator layer); the "what actually fails" exercise — region-level vs. AZ-level vs. single-service failure; and the operational cost of testing failover on a schedule rather than trusting it because it was designed correctly once.

☐ 6. How do you keep an architecture roadmap honest against the reality of unplanned production firefighting eating your team's time?

> Track toil / unplanned work as its own visible metric (percent of sprint capacity), protect a fixed allocation for roadmap work even when it's uncomfortable to defend, and treat a chronically-exceeded firefighting budget as itself an architecture signal — it usually means the roadmap's top priority should be the thing causing the firefighting, not whatever was originally planned.

3 SRE Principles: SLI/SLO/Error Budgets

☐ 1. Your service is within its error budget for the month, but customers are complaining about degraded experience. What's actually going on, and what do you do?

> Classic SLI-doesn't-match-reality gap. Likely causes: the SLI measures the wrong thing (availability but not latency, or a synthetic check that doesn't mirror the real request path), a customer segment isn't represented in the aggregate (a region or tenant averaged out), or the SLO threshold is simply too loose. Pull customer-reported incidents, correlate against dimensions you're *not* currently slicing by (region/tenant/endpoint), and treat this as an SLI-redesign trigger — not a "budget says we're fine" dismissal.

☐ 2. How do you set an SLO for a brand-new service with no historical data?

> Start from user expectation, not measured data: use comparable services' baselines, negotiate an initial conservative target with product/customers, instrument from day one, and explicitly revisit after 4–6 weeks of real traffic. Call it a "provisional SLO" so no one anchors hard on a number that was chosen blind.

☐ 3. Your error budget is fully burned for the quarter with two weeks left. Product wants to ship a risky feature anyway. What do you do?

> This is the core test of whether the error budget is a real policy or theater. If a feature-freeze-on-exhaustion policy exists, enforce it and escalate to the pre-agreed decision-maker for a tiebreak rather than unilaterally overriding — or unilaterally blocking without escalating. If no formal policy exists, that absence *is* the finding — surface it and push to get one written before the next cycle.

☐ 4. Two teams both claim ownership of a failing dependency during an incident. How do you drive fault isolation quickly?

> Don't resolve ownership first — resolve customer impact first (mitigate, rollback, or failover). Run fault isolation in parallel using distributed tracing to establish the actual failure boundary with evidence, not assertion, then assign ownership based on what the trace data actually shows.

☐ 5. How would you design a golden-signal dashboard for a service you've just inherited with zero existing observability?

> Cover RED/USE: Latency (p50/p95/p99 — never just average), Traffic (request rate), Errors (rate plus error-budget burn), and Saturation (CPU/mem/connection-pool/queue depth). Instrument from the edge (load balancer) inward first, so you get a customer-true view before internal service-level views.

☐ 6. Explain multi-window, multi-burn-rate alerting and why you'd use it over a simple threshold alert.

> A fast-burn window (short lookback, high burn-rate threshold) pages immediately for fast-moving outages. A slow-burn window (longer lookback, lower threshold) catches slow degradations a short window would miss, or that would just flap noisily. Combining both reduces missed slow incidents *and* alert fatigue from short-window flapping — a single threshold alert can't do both well.

☐ 7. A postmortem keeps surfacing 'human error' as root cause across multiple incidents. How do you push the analysis further?

> "Human error" is almost always a symptom of a systemic gap — missing guardrails, an unclear runbook, alert fatigue, or a tooling trap that makes the mistake easy. Push the 5-whys past the human action to the system condition that allowed it, and track repeat "human error" postmortems as a leading indicator that tooling/process investment is overdue.

4 Incident Management & Postmortems

☐ 1. Walk through your incident command process from page to resolution for a P1.

> Ack/triage within a target time; declare severity and assign IC/scribe roles explicitly rather than ad hoc; stabilize-first mindset — mitigate before you fully root-cause; maintain a running timeline in the incident channel; a clear handoff protocol for long-running incidents; and a blameless postmortem within a set window after resolution.

☐ 2. A postmortem review turns adversarial — another team feels blamed. How do you keep it blameless while still being rigorous about root cause?

> Separate "what happened" (facts, timeline, systemic conditions) from "who's at fault" explicitly in the document's structure. Frame action items as system/process changes, not individual behavior changes. If a team feels singled out, redirect to "what would have made this easier to avoid, regardless of who touched it."

☐ 3. How do you handle an incident where the root cause is genuinely unclear even after mitigation — do you still run a postmortem?

> Yes. Document what's known, what's unknown, and the follow-up investigation as an explicit action item with an owner and a deadline. Don't let "cause unknown" quietly close the loop — an unresolved unknown is itself a recurrence risk.

☐ 4. Customer executives are on a bridge call demanding an ETA during a live P1 you don't have full visibility into yet. How do you handle it?

> Give a confidence-bounded answer, not a guess dressed as certainty: state what's known, what's being tried, and a time-boxed next-update commitment ("next update in 15 minutes regardless of progress") rather than promising a resolution time you can't back. Route stakeholder management to a comms lead if one exists, so the IC can stay heads-down on mitigation.

☐ 5. You find during an incident that a runbook you're relying on is out of date and gives wrong steps. What do you do in the moment, and afterward?

> In the moment: don't blindly follow it — verify against current system state before executing anything destructive or irreversible. Afterward: fix the runbook the same day as an immediate action item, and treat "how did this go stale?" as its own follow-up — it usually points to a missing ownership or review cadence for runbooks generally.

☐ 6. How do you decide what counts as a 'near miss' worth reviewing versus normal operational noise?

> A near miss is any event where the outcome depended on luck or an individual catching something rather than a systemic control — an alert that almost didn't fire, a rollback that almost went to the wrong environment. Review these with a lighter-weight process than a full postmortem, but review them — they're the cheapest signal you'll get for preventing a future real incident.

5 AWS & Multi-Region Reliability

☐ 1. Design a multi-region active-active architecture for a stateful service on AWS. What's the hardest part?

> The hardest part is almost always data consistency/replication, not compute — compute is stateless and trivially multi-region. The database/cache layer forces a choice between eventual consistency (global-replicated databases or global tables) with conflict/staleness handling, or a single write region with read replicas elsewhere — simpler, but not truly active-active for writes. Walk through that latency/consistency trade-off explicitly rather than hand-waving it.

☐ 2. An AWS region-level event takes down one of your regions. What's your failover sequence, and what cutover time would you target?

> Cover: health-check-driven DNS failover vs. an anycast/accelerator-style routing layer; the RTO/RPO you'd design to, which differs by data store (a global-database-style setup can target roughly sub-minute RTO / sub-second-to-low-single-digit-second RPO — know a concrete figure cold if asked); and the operational discipline of testing failover on a schedule (game days) rather than trusting it because it was designed correctly once.

☐ 3. You're seeing intermittent 5xx errors from a load-balancer-fronted EKS service, but pods look healthy in kubectl. Where do you look?

> Check target-group health-check config against the readiness probe for a mismatch, connection draining/deregistration delay during rolling deploys, the load balancer's access logs for the actual error codes and latency, and whether the issue is client-side (connection reuse against a stale/rotated pod IP) rather than server-side at all.

☐ 4. How do you approach cost vs. reliability trade-offs when a customer or product team pushes back on the cost of your proposed HA design?

> Translate reliability into business risk in their terms — expected cost of an outage (SLA credits, churn risk, support load) versus the incremental infra cost — and offer tiered options (warm-standby vs. full active-active) so it becomes a negotiated trade-off rather than a binary yes/no.

☐ 5. Explain the difference between an AZ failure and a region failure in terms of what your architecture needs to survive each — and give an example of a service that's AZ-resilient but not region-resilient.

> A multi-AZ deployment (a multi-AZ managed database, an autoscaling group spanning three AZs) survives a single data-center-level failure, but shares the region's control plane, DNS, and often the same VPC — so a region-level event (or a regional service degradation like identity/token services) still takes it down. True region resilience requires independent deployment plus data replication into a second region.

☐ 6. A cost review flags that your multi-region setup doubles infra spend for a service that rarely needs it. How do you defend or adjust the design?

> Tie the design back to the actual SLA/customer commitment driving it. If the commitment doesn't justify active-active, propose a cheaper pattern — pilot-light or warm-standby — that still meets the required RTO/RPO at lower steady-state cost, and be willing to concede the design if the business case genuinely doesn't support it.

☐ 7. What's your process for validating that IAM permissions across a multi-account AWS setup follow least privilege, at enterprise scale?

> Organization-level guardrails (SCPs) as the ceiling, not per-resource enforcement; permission boundaries for delegated admin; periodic access-analysis reviews to catch unused permissions; and treating IAM policy changes like code — PR review and drift detection — rather than console click-ops.

6 Kubernetes / EKS — Tricky Scenarios

☐ 1. A Deployment rollout is stuck at 'Pending' for new pods, but the old pods are still Running and serving traffic. What's your triage order?

> Check pod events first (describe the pod) for scheduling failures — insufficient node resources, node-selector/affinity/taint mismatch, a stuck PVC binding, or a PodDisruptionBudget blocking eviction of old pods so new ones can't schedule under resource pressure. Since old pods still serve traffic, this isn't customer-impacting yet — don't panic-rollback before reading the events.

☐ 2. You scale a Deployment down and back up quickly, and a service starts throwing connection errors even though pods show Running. Why?

> Running is not the same as Ready — a pod can pass liveness but not yet pass readiness while it finishes warming up dependencies. If it's added to Service endpoints prematurely, or if client-side connection pooling is caching a now-dead pod IP from before the scale-down, requests fail. Check readiness-probe timing/thresholds and endpoint-slice update lag.

☐ 3. A ConfigMap change doesn't take effect after you update it, even though the API shows the new value. Why, and how do you fix it?

> Pods don't auto-reload ConfigMap changes mounted as environment variables (only volume-mounted files eventually sync, and even then the app must watch and reload). You need a rolling restart to pick up env-var-sourced config, or a reloader/checksum-annotation pattern to trigger one automatically on ConfigMap change.

☐ 4. A namespace shows zero visible resource issues, but pods are getting OOMKilled. Metrics show usage under the limit. What's happening?

> OOMKilled is enforced by the cgroup memory limit at the moment of allocation, which can spike faster than the metrics sampling interval captures — the process may briefly exceed the limit between samples. Also check whether a memory limit is even set (if only requests exist, there's no hard ceiling) and whether a JVM-based container is sizing its heap against host memory rather than the container's cgroup limit.

☐ 5. You need to safely drain a node running a StatefulSet-backed database pod during a cluster upgrade. What could go wrong, and how do you prevent it?

> Risks: a PodDisruptionBudget blocking the drain indefinitely if minAvailable is set too strictly for a single-replica StatefulSet, or the drain succeeding but the pod rescheduling to a node without access to the volume's original AZ. Prevent by verifying the PDB allows a controlled disruption, confirming the storage class supports the target AZ, and draining one replica at a time with health verification between.

☐ 6. A managed EKS cluster upgrade goes wrong mid-way — some nodes are on the old version, some on new, and pods are failing cross-version. How do you stabilize first, then fix properly?

> Stabilize: pause further rollout, confirm which workloads are actually failing — often an API deprecation between versions, not just node skew — and consider cordoning problem nodes rather than making further changes blind. Fix: roll node groups the rest of the way forward, since Kubernetes doesn't support moving backward past a version once the control plane is upgraded, using a rolling or blue/green node-group strategy — and check the target version's deprecated-API list *before* the upgrade next time.

☐ 7. How would you debug a service with no Kubernetes-visible errors, but customers report requests are slow only intermittently?

> This is a distributed-tracing problem, not a kubectl problem. Use tracing spans across the request path to find where latency accumulates — DNS resolution, connection-pool exhaustion, a specific downstream dependency, GC pauses — correlate against node-level saturation (noisy-neighbor CPU throttling under cgroup limits), and check whether it correlates with autoscaler scale-out events (cold-start latency on newly scheduled pods).

7 Terraform / IaC — Tricky Scenarios

☐ 1. A teammate ran terraform apply and it silently destroyed and recreated a production database instance instead of just applying a tag change. What likely happened, and how do you prevent recurrence?

> Almost always a "forces replacement" change buried in the plan output that wasn't read carefully — a change to an immutable attribute (identifier, certain engine-version jumps, subnet group in some configs) forces destroy/recreate instead of an in-place update. Prevention: mandatory plan review with an explicit check for "must be replaced" lines before any prod apply, ideally enforced by a CI gate that fails on unexpected destroy actions, plus lifecycle prevent_destroy on stateful resources.

☐ 2. Two engineers both applied changes to the same Terraform workspace within minutes of each other and now the state looks inconsistent. What's your recovery process?

> Points to a missing or misconfigured state lock. Recovery: don't guess-fix — pull the state, diff it against actual infra reality with a plan (which will surface drift), reconcile manually resource-by-resource if needed, and fix the locking gap immediately so it can't recur.

☐ 3. You need to rename a resource in your Terraform config without Terraform destroying and recreating the underlying infrastructure. How?

> Use a moved block (Terraform 1.1+) or the state-mv command for older syntax — this tells Terraform the resource's address changed without treating it as a brand-new resource. High-value gotcha to know cold: getting it wrong is a silent destroy/recreate, not an error.

☐ 4. A count-based resource list had an item removed from the middle, and now Terraform wants to destroy/recreate several unrelated resources. Why, and what's the fix?

> count indexes by position, so removing a middle element reindexes everything after it, and Terraform sees those as different resources at different indices. Fix: migrate to for_each keyed by a stable identifier — name or id — so removing one element doesn't reindex the rest.

☐ 5. You mark a variable sensitive=true in Terraform, but a teammate finds the value in plaintext in the remote state file. Is that expected?

> Yes, historically — sensitive=true only suppresses the value from CLI/plan output, not from the state file itself; state must be encrypted at rest and access-controlled regardless. Newer ephemeral/write-only attribute support in recent Terraform versions is aimed at keeping certain values out of state entirely — worth mentioning as the fix direction, but the core practice is still: treat state as sensitive data no matter what.

8 Docker & CI/CD Gotchas

☐ 1. A container that's supposed to be stateless keeps accumulating disk usage across restarts in the same environment. Why might restarting not actually be clearing it?

> If it's a plain container restart, the same writable filesystem layer persists — writes since the image was built remain. Only recreating the container from the image (removing and re-running it, or in Kubernetes, pod recreation) resets the writable layer. Also check for a mounted volume that's outliving the container by design.

☐ 2. Your image uses the 'latest' tag and a deploy pulls a different image than what was tested in staging. What's the systemic fix, not just 'don't do that'?

> Pin to immutable digests or specific tags in every environment, set the image pull policy deliberately (never rely on 'latest' plus an ambiguous pull policy), and enforce it via a policy check — an admission controller or CI lint — that rejects 'latest' in any deploy manifest. Relying on team discipline alone doesn't scale.

☐ 3. A CI pipeline shows green, but the deployed service is broken. What pipeline design gaps does this usually point to?

> Usually: tests don't cover the actual failure mode (missing integration/smoke test against the real deploy target), a pipefail-equivalent gap where a failing step inside a multi-command shell stage didn't actually fail the job, or the pipeline validates the build artifact but has no post-deploy health check or automatic rollback gate.

☐ 4. You discover a secret was accidentally committed and pushed to a shared repo, and the pipeline may have already deployed with it embedded. What's your immediate response sequence?

> Rotate the secret immediately — don't wait to finish cleanup, assume compromise the moment it was pushed. Remove it from the pipeline/deploy artifacts and force a redeploy with the new secret. Deal with git-history scrubbing as a secondary step, since the exposure already happened regardless of history cleanup.

☐ 5. Two deploys to the same environment triggered by concurrent merges race, and the second half-overwrites the first. How do you prevent this at the pipeline level?

> Enforce a deploy concurrency lock or queue at the environment level so a second deploy waits or cancels-and-restarts rather than racing, and make deploys idempotent/atomic — a single release object promoted, not partial file overwrites — so a partial failure never leaves a mixed-version state.

9 Observability: Prometheus/Thanos/Splunk/CloudWatch/OTel/ELK

☐ 1. The JD names Thanos specifically — why would a team add Thanos on top of Prometheus rather than just running Prometheus with longer retention?

> Prometheus is built for local, relatively short-retention time-series storage on a single node — pushing retention out to months/years bloats local disk and hurts query performance, and a single Prometheus has no built-in HA or cross-cluster global view. Thanos adds three things Prometheus alone doesn't: long-term cheap storage in object storage via the Sidecar uploading TSDB blocks, a global query view across multiple Prometheus instances/clusters through the Querier, and deduplicated high availability when running redundant Prometheus pairs.

☐ 2. Explain what each core Thanos component does, at a level you'd want cold in an interview.

> **Sidecar** — runs alongside Prometheus, uploads TSDB blocks to object storage and serves recent data via the Store API. **Store Gateway** — serves the historical data already in object storage back to queries. **Compactor** — downsamples, compacts, and applies retention on object-storage data (must run as a singleton). **Querier** — the global PromQL entry point, fanning out to Sidecars and Store Gateways and deduplicating HA pairs. Optionally: Ruler for global alerting/recording rules, Receiver for remote-write ingestion.

☐ 3. A Grafana dashboard querying through Thanos is dramatically slower for a 1-year range than a 1-day range, even after downsampling should have kicked in. What do you check?

> Check whether the Compactor is actually running and keeping up — a stalled or singleton-violated Compactor means raw, undownsampled blocks are still being scanned at full resolution. Check whether the query is actually eligible for auto-downsampling on that time range. Check the Store Gateway's index cache sizing — a cold cache on a big range causes slow object-storage index scans regardless of downsampling.

☐ 4. How would you design an alert-routing strategy in Alertmanager to avoid paging the whole team for every warning-level alert?

> Severity-based routing trees — critical pages on-call immediately, warning routes to a chat channel or ticket queue; grouping and inhibition rules so a root-cause alert suppresses its downstream symptom alerts; and time-based routing so low-severity noise doesn't page overnight when it can safely wait.

☐ 5. You're asked to add distributed tracing to a service that currently only has metrics and logs. What's your rollout approach, and what does tracing give you that the others don't?

> Traces give you the causal path of a single request across services — something aggregate metrics and per-event, hard-to-correlate logs can't show on their own — which is the payoff for root-causing cross-service latency and pinpointing exactly where a failure originated in a call chain. Rollout: start with auto-instrumentation at the edge/gateway for immediate wins, propagate trace context through the highest-traffic paths first, and treat full manual span instrumentation of every internal call as a later-phase investment, not day one.

☐ 6. Log-ingestion costs are becoming a problem at scale. How do you reduce cost without losing the ability to investigate incidents?

> Tier data by value: keep high-cardinality/high-value logs (errors, security events) at full retention and full indexing, route low-value verbose logs (debug-level, health-check noise) to a cheaper tier or filter them at the source before ingestion, and pre-aggregate what's usually queried in dashboards via summary/scheduled searches instead of scanning raw data every time.

☐ 7. How do you build a dashboard that a non-technical stakeholder — say, a customer-success lead — can actually use, versus one built for engineers?

> Lead with business-relevant panels (SLA compliance percentage, active incidents, trend over time) rather than raw metric panels, use plain-language panel titles instead of query-derived defaults, and favor a more guided dashboard-builder layout over a dense, ops-style grid meant for engineers scanning many signals at once.

☐ 8. Alarms are firing correctly but engineers are ignoring them due to alert fatigue. How do you fix the underlying problem, not just mute alerts?

> Audit existing alarms for signal quality — retire or raise the threshold on alarms with a high false-positive rate, consolidate related alarms into composite alarms so one root cause doesn't fire five separate pages, and tie every remaining alarm explicitly to a runbook so an engineer always has a clear next action rather than an ambiguous "something's wrong" ping.

☐ 9. How does an ELK-style stack's role differ from a commercial platform like Splunk's, in an environment where you might use both — when would you reach for which?

> They overlap heavily in practice — both are log aggregation and search — but the open-source stack is typically the cheaper, self-hosted choice for high-volume, less business-critical logs, while a commercial platform's licensing cost is often justified for compliance-sensitive or business-stakeholder-facing use cases, given its dashboarding/alerting UX and enterprise support. If both coexist in one org, it's usually a legacy-coexistence or cost-tiering decision rather than a clean split by function.

10 Linux, Networking & Security

☐ 1. A service intermittently can't resolve a downstream hostname under load, but works fine at low traffic. What do you check?

> Likely a DNS resolver or connection-tracking exhaustion issue under high concurrent lookup volume — check ephemeral-port exhaustion on the resolver path, whether the client caches negative DNS responses too aggressively, and at the container level, a shared node's resolver search-domain settings causing extra unnecessary lookups per request, a classic Kubernetes DNS-amplification gotcha.

☐ 2. How would you explain the client-server request path at the socket/kernel level for a simple HTTPS request?

> DNS resolution → TCP three-way handshake (SYN / SYN-ACK / ACK) → TLS handshake (certificate exchange, key negotiation) → the actual HTTP request/response over the established connection → connection teardown or keep-alive reuse. Know where each step commonly breaks in production: DNS timeout, SYN-backlog exhaustion under load, certificate expiry, connection-pool exhaustion.

☐ 3. A production host is hitting file-descriptor limits and unrelated processes start failing. Walk through your diagnosis and fix.

> Diagnose: check the shell/process ulimit, the actual running process's ceiling from its proc-level limits, and use a tool like lsof to see what's genuinely holding descriptors open — usually a connection leak rather than a real need for more capacity. Fix: raise the limit as a stopgap only if legitimately near capacity, but treat FD exhaustion primarily as a leak-hunting exercise, not a limits-tuning one.

☐ 4. Design a network security review checklist for a new service before it goes to production in a regulated environment.

> Least-privilege security groups/network ACLs (no open-to-the-world rules on anything but the public load balancer), encryption in transit everywhere internal too — not just at the edge — and at rest, secrets never visible in plain env vars via container inspection tools (use a secrets manager), IAM role scoping per service rather than a shared broad role, and logging/audit coverage sufficient to reconstruct an incident after the fact.

☐ 5. Explain the CIDR math you'd need to correctly size a VPC subnet for a Kubernetes cluster that needs to scale to several hundred pods, and what happens if you get it wrong.

> Each pod typically gets its own VPC IP in AWS's native CNI (unlike overlay-network CNIs), so subnet sizing must account for max-pods-per-node times node count, not just node count. Undersizing means new pods fail to schedule with IP-exhaustion errors once the subnet's free IPs run out — usually discovered painfully during a scale-up event, not at design time. Mitigations: a dedicated larger CIDR for pod IPs, secondary CIDR ranges, or prefix-delegation CNI mode to conserve IPs.

☐ 6. How would you approach a security finding that a service is using overly broad IAM permissions, without breaking production while you fix it?

> Don't cut permissions blind — first observe actual usage over a representative period to build the real permission set needed, deploy the tightened policy to a non-prod or canary path first, and roll out with monitoring for access-denied errors before fully cutting the old broad policy, rather than a hard cutover.

11 Automation & Programming (Python/Go)

☐ 1. The JD wants Python, Go, or equivalent for building core data-platform capabilities — how do you talk about your programming depth if your background is stronger in scripting/automation than in shipping production services?

> **Bring your own** — be honest about the actual shape of your experience (automation scripts, IaC, CI/CD glue code vs. long-running services), but connect it to what's genuinely transferable: concurrency awareness, error handling, and testing discipline carry across from any language. Offer a concrete example of the most "software-engineering-shaped" piece of code you've written, not just a one-off script.

☐ 2. Give an example of when you'd choose to build a custom automation tool versus adopting an existing one.

> Default to adopting for anything that's a solved, non-differentiating problem — generic drift detection, generic alerting. Build custom only when existing tools don't fit a genuine constraint specific to your environment (a nonstandard multi-cloud reconciliation need, a legacy system with no API), and the maintenance cost of owning it is clearly less than the cost of working around a mismatched tool long-term.

☐ 3. How would you structure a Python automation script meant to run unattended in production — say, a reconciliation or cleanup job — so it fails safely?

> Idempotency, so it's safe to re-run after a partial failure; dry-run as the default mode with an explicit flag required to execute destructively; structured logging so a failure is diagnosable without re-running; alerting on failure rather than a silent exit; and a blast-radius guard for anything destructive — refusing to act on more than a set percentage of records in one run.

☐ 4. What's your approach to choosing between declarative IaC and imperative Python/SDK scripting for a given automation task?

> Declarative IaC for anything representing durable infrastructure state you want drift-detected and version-controlled. Imperative scripting for one-off operational tasks, data migrations, or orchestration logic that doesn't map cleanly to "desired state" — a phased cutover with conditional logic, for example. Mixing them carelessly, where a script mutates IaC-managed resources outside the IaC tool, is a common source of drift/state-mismatch incidents.

☐ 5. How do you test infrastructure automation code before it touches production, given you can't easily unit-test 'does this actually reconfigure a real cloud resource'?

> Layer it: static validation and plan-diff review catch syntax/logic errors cheaply; a real but isolated staging or sandbox account exercises actual API behavior; and for scripts, mock the SDK calls for fast unit tests of logic while reserving a small set of integration tests against real non-prod resources for the parts that matter most.

12 Customer & Cross-Functional Engagement

☐ 1. A customer's feedback about a recurring reliability issue conflicts with what your internal metrics say — 'everything's green.' How do you reconcile that and turn it into action?

> Treat the customer report as a signal your instrumentation might be blind to, not as noise to dismiss. Dig into what the customer actually experiences — their specific workflow, region, or usage pattern — versus what your SLI aggregates across all customers. If there's a genuine instrumentation gap, fix the SLI, not just the one customer's symptom.

☐ 2. How do you translate a vague piece of customer feedback — 'the platform feels slow sometimes' — into an actionable engineering insight?

> Get specificity through structured follow-up: when, which action, how often, does it correlate with time of day or data volume. Instrument to capture the exact interaction if it's not already traced. Reproduce with data before committing engineering time — vague feedback often maps to one specific, findable bottleneck once you narrow the "sometimes."

☐ 3. You're in a meeting with product management and they want a reliability commitment — an SLA number — that you're not confident the current architecture can support. How do you handle that conversation?

> Don't just say no. Show the gap with data — current measured reliability versus the ask — and the cost/timeline to close it, so it becomes a resourcing/trade-off conversation rather than a standoff. Agreeing to an SLA you know you can't hit just moves the failure downstream to a worse, more public moment.

☐ 4. How do you communicate a complex technical root cause to a non-technical customer-facing stakeholder, like an account manager, who needs to explain it to the customer?

> Strip it to: what broke, in customer-impact terms rather than internals; why it won't recur, the fix in one sentence; and what changed to detect it faster next time. Skip internal jargon and the full technical chain unless specifically asked — the stakeholder needs a defensible, honest, simple narrative, not the full RCA document.

☐ 5. A customer-facing team wants to bypass your normal escalation process because their customer is high-value, essentially asking for special-case handling on every incident. How do you handle it constructively?

> **Bring your own** — general framework: acknowledge the business pressure honestly, but redirect toward making the standard process fast enough that bypassing it isn't necessary — a genuinely fast, pre-defined escalation tier for top-tier accounts — rather than ad hoc exceptions, since ad hoc bypassing usually degrades process discipline for everyone over time.

13 Mentoring & Technical Leadership

☐ 1. How do you mentor a mid-level engineer who's technically strong but struggles with production ownership or on-call confidence?

> **Bring your own** — framework: pair on live incidents rather than only postmortems, to build pattern recognition in real time; give them increasing incident-command responsibility with you as backup; and separate a technical-skill gap (needs training) from a confidence gap (needs reps and psychological safety), since the interventions differ.

☐ 2. You disagree with a technical decision another senior engineer or your manager has made. How do you handle that without undermining the team?

> Raise it privately first, with data and reasoning, not in front of the team. If you're overruled after a genuine hearing, commit to the decision publicly — disagree-and-commit — rather than passive-aggressively signaling doubt, unless it's a genuine safety or reliability red line, in which case escalate explicitly rather than quietly resisting.

☐ 3. How do you scale your own knowledge across a team so you're not the single point of failure for a critical system?

> Documentation that's actually used — tied to runbooks exercised in game days, not written and forgotten — pairing and rotation on the critical system's on-call and maintenance work, and deliberately not being the only approver on changes to it. Treat a "bus factor of one" itself as a reliability risk to be tracked and reduced.

☐ 4. How do you give technical feedback on a junior engineer's design or code that has real problems, without discouraging them?

> Separate the idea from the person explicitly, lead with what's working before what needs to change, ask questions that guide them to find the issue themselves where possible rather than just handing them the fix, and calibrate directness to how much production risk the issue actually carries — a style nitpick and a data-loss risk don't get the same tone.

14 Behavioral — Tricky Situational

☐ **1. Tell me about a time you made a call under pressure that you later found out was wrong. How did you handle it?**

> **Bring your own** — prepare: what the call was, what information you had (and didn't) at the time, when and how you learned it was wrong, and specifically what you changed afterward — process, not just "I'll be more careful." Interviewers are testing for ownership and systemic learning, not perfection.

☐ **2. Describe a situation where you had to say no to a stakeholder's request. What happened?**

> **Bring your own** — prepare one real example with the actual trade-off you surfaced, how you offered an alternative rather than a flat no, and the outcome — including if it went poorly. An honest "it created friction, here's what I'd do differently" answer is stronger than a too-clean success story.

☐ **3. How do you prioritize when everything is labeled 'urgent' by different stakeholders simultaneously?**

> Separate genuinely time-sensitive work (a customer-facing outage, a security exposure) from urgent-sounding-but-not (a deadline set without a real external constraint), get an explicit ranking from whoever has authority to arbitrate across competing asks rather than deciding unilaterally when it crosses team boundaries, and communicate the resulting sequencing transparently so no one is surprised later.

☐ **4. Tell me about a time a project or initiative you owned failed or was cancelled. What did you take away from it?**

> **Bring your own** — resist over-polishing this one; a real failure with genuine lessons reads far better than a disguised success story. Prepare the actual reason it failed — scope, timing, priority shift, a technical wall — and one concrete thing you do differently now as a result.

☐ **5. How do you handle being wrong in front of your team during a live incident?**

> **Bring your own** — general framework: acknowledge it plainly and immediately rather than quietly correcting course, since visible ownership from a senior person sets the norm for the whole team's psychological safety during incidents, then move straight back to mitigation rather than a lengthy explanation mid-incident.

☐ **6. You're asked to take on a project outside your comfort zone — deeper Go or data-platform engineering than you've done before — to fill a gap on the team. How do you approach it?**

> **Bring your own** — general shape: be honest about the starting gap rather than overstating readiness, propose a concrete ramp-up plan (pairing, a bounded first task, a defined check-in cadence), and frame it as growth you want, if that's genuinely true for you, rather than something you're reluctantly accepting.